Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Iterator (davranışsal kalıp) (ekrandan, 00:06:10)
  - Amaç: parçalardan oluşan bir bileşenin parçalarını, bileşenlerin iç yapısını açığa çıkarmadan, istenen bir biçimde dolaşmak. Örnek: bir veri yapısı içinde dolaşmak.
  - Sorun: veri yapısının arayüzünü birçok farklı dolaşım türüyle şişirmek istemeyiz. Hoca örnek olarak ikili ağacı verdi: LDR (Left-Data-Right) gibi sol, veri ve sağ harflerinin sıralanmasıyla $3! = 6$ dolaşım türü olur. Bağlı listede de tek ya da çift yönlü olma ve dairesel olma durumuna göre farklı dolaşımlar vardır.
  - Java'da her koleksiyondan bir `Iterator` istenebilir, ama bu nesne kalıba tam uymaz; daha basittir. Basit, kötü demek değildir. KISS (Keep It Simple, Stupid) ilkesi gereği Java'nın `Iterator`'ı işe yarıyorsa o kullanılır.
- Iterator: birinci çözüm (ekrandan, 00:10:00)
  - `List` (`Count()`, `Append(Element)`, `Remove(Element)`, ...) ve onu bilen `ListIterator` (`First()`, `Next()`, `IsDone()`, `CurrentItem()`, `index`).
  - Güçlü yönü: veri yapısı ile dolaşım biçimi birbirinden soyutlanmıştır.
  - Zayıf yönü: belli bir veri yapısı ile belli bir dolaşım biçimi arasında bağlılık oluşur. Bunu azaltmak için genel bir liste arayüzü ve genel bir dolaşım arayüzü oluşturulabilir. Hoca bunun Bridge kalıbını hatırlattığını söyledi.
- Iterator: ikinci çözüm (ekrandan, 00:12:10)
  - Soyut `AbstractList` (`CreateIterator()`, `Count()`, `Append(Item)`, `Remove(Item)`) ve soyut `Iterator` (`First()`, `Next()`, `IsDone()`, `CurrentItem()`).
  - `List` ve `SkipList` alt sınıflarının karşılığı `ListIterator` ve `SkipListIterator`'dır. `Client` yalnız iki soyut tiple çalışır, kendisine hangi yapı ve gezgin verilirse onu dolaşır.
  - Somut yapı ile somut gezgin arasındaki bağlaşım kaçınılmazdır, ama istemci bu ayrıntılardan korunur.
- Iterator gerçekleme ayrıntıları (ekrandan, 00:14:00)
  - Dolaşırken parçaların değiştirilmesine izin verilecek mi? İzin verilmeyecekse veri yapısı kendisinin bir klonunu gezgine verir.
  - Dolaşım algoritmasını kim gerçekleyecek, nesneler birer birer mi hepsi birden mi verilecek? Ya istemci gezgine komut verme sırasını belirler ve parçaları tek tek okur, ya da bir dolaşım isteği verir ve gezgin parçaları bir dizide döndürür.
  - Sıra önemliyse gezginin ek metotlarıyla sağlanabilir: anahtar-değer tablosunda anahtarların artan ya da azalan sırası, ağaçta LDR ya da DLR gibi. Farklı dolaşım biçimleri Strategy kalıbıyla da gerçeklenebilir.
- Iterator örneği: online yemek siparişi (ekrandan, 00:17:00; kaynak Head First Design Patterns)
  - Biri kahvaltılıkta, biri yemekte uzman iki firma satın alınır. Menü elemanı (`MenuItem`) gerçeklemesi aynıdır ama menüler farklı gerçeklenmiştir: `DinerMenu` dizi, `PancakeHouseMenu` `ArrayList` kullanır.
  - Ortak kod olmazsa her yeni firma için menü sunma koduna yeni bir blok eklenir. Vejetaryen menü için iki menü ayrı ayrı dolaşılmalıdır.
  - Acemi çözüm (`WaitressQuickAndDirty`): garson iki somut menü tipinde iki alan tutar, her menüyü kendi veri yapısına göre ayrı döngüyle dolaşır; yinelenen kod oluşur.
  - Kalıpla çözüm: ortak `Menu` arayüzüne `java.util.Iterator<MenuItem> createIterator()` eklenir (ekrandan, 00:20:00). `DinerMenuIterator` ve `PancakeHouseMenuIterator` yazılır. `Waitress` menüleri `Menu` tipinde tutar ve tek bir `printMenu(Iterator)` me-

toduyla hepsini dolaşır (ekrandan, 00:50:00).

- Memento (ekrandan, 01:08:45)
  - Amaç: bir nesnenin durum bilgisini sarmalama ilkesini bozmadan elde etmek. Böylece nesne ikincil saklama ortamına yazılabilir, geri yüklenebilir, ağ üzerinden aktarılabilir; geri alma (undo) düzeneği kurulabilir. Durum bilgisinin yalnız bir kısmının saklanması da yeterli olabilir.
  - Yapı (ekrandan, 01:10:00): `Originator` (`state`, `CreateMemento()` → `return new Memento(state)`, `SetMemento(Memento m)` → `state = m->GetState()`), `Memento` (`state`, `GetState()`, `SetState()`), `Caretaker` mementoları saklar.
  - Etkileşim (ekrandan, 01:13:00): `Caretaker` `CreateMemento()` çağırır; `Originator` yeni bir `Memento` oluşturur, `SetState()` ile durumu ona yazar ve döndürür. Geri dönmek gerektiğinde `Caretaker` `SetMemento(aMemento)` çağırır, `Originator` `GetState()` ile durumu alır.
- Memento'yu korumak
  - Originator'a sahte bir memento verilememeli, kimse mementoyu bozamamalı (tamper, invalidate). Mementonun metotlarına yalnız originator erişebilmeli; caretaker bile erişememeli.
  - C++: memento metotları `private` yapılır, `Originator` `Memento`'nun `friend`'i ilan edilir.
  - Java, birinci yol: memento metotları paket düzeyinde bırakılır, `Memento` ve `Originator` aynı pakete, `Caretaker` başka pakete konur. Daha az denetim sağlar.
  - Java, ikinci yol: `Memento`, `Originator`'ın `public` iç sınıfı yapılır, metotları `private` olur. Caretaker memento nesnesini tutabilir ama değiştiremez. Durum sınıfı klonlanabilir yapılır ve `createMemento()` durumun klonunu verir.
- Memento kod gösterimi (ekrandan, 01:30:00, 01:40:00)
  - Paketler: `dp.memento` ve `dp.mementoOuter` (paket düzeyi çözüm), `dp.mementoV2.innerClass` ve `dp.mementoV2Outer` (iç sınıf çözümü). Caretaker durumu "ilk durum" yapar, memento alır, durumu değiştirir, mementoyu geri verir; çıktı "ilk durum" olur.
  - Kırma denemesi: başka bir `Originator`'dan ("sahte durum") alınan memento asıl nesneye verilebildi. Mementonun iç bütünlüğü bozulmadı; hoca bunun geçerli bir işlem sayılıp sayılmayacağını yoruma bıraktı.
  - Engelleme önerileri: originator kurucuda rastgele bir `long` kimlik (ya da hash kodu, ya da seri numarası) üretir ve mementoya yazar; `setMemento` kimlik uyuşmazsa atamayı yapmaz. Ya da mementoları caretaker yerine originator'ın kendisi saklar.
  - `clone()` `return this` yapacak biçimde değiştirilince geri dönüş çalışmadı: memento ve nesne aynı durum nesnesini paylaştı.
- Memento örneği: belge işleme programında geri alma ve yineleme (ekrandan, 01:48:45)
  - `Controller` 1 → * `Document`; `Document`: `currentState : DocumentMemento`, `states : List<DocumentMemento>`, `position : int`, `undo()`, `redo()`, `anyOperation()`. `Controller`'ın `DocumentMemento` ile hiçbir ilişkisi yoktur.
  - `undo()` konumu bir azaltıp o konumdaki durumu geçerli yapar, `redo()` bir artırır, durumu değiştiren her işlem durumu listeye ekler. Konum negatife ya da listenin dışına düşebilir; bu denetlenmelidir.
- Observer (ekrandan, 02:10:00)
  - Amaç: bir nesnenin durumu değiştiğinde onunla ilişkili bütün nesnelerin haberdar olmasını ve güncellenmesini sağlamak. İlişkiler çalışma anında kurulup bırakılabilmelidir. Örnek: hesap tablosundaki veri değişince ona bağlı grafikler de güncellenir.
  - Yapı: soyut `Subject` (`Attach(Observer)`, `Detach(Observer)`, `Notify()` → `for all o in observers { o->Update() }`), `ConcreteSubject` (`GetState()`, `SetState()`, `subjectState`), soyut `Observer` (`Update()`), `ConcreteObserver` (`observerState = subject->GetState()`). `Subject`'ten `Observer`'a bire-çok sahiplik vardır.
- Observer gerçekleme ayrıntıları (ekrandan, 02:12:00, 02:20:00)
  - Pull model: gözlenen yalnız "değiştim" der, gözleyiciler ilgili durumu gözlenenden alır. Bağlantıyı azaltır, gözleyici durumun yalnız istediği kısmını alabilir; ama nelerin değiştiğini gözleyici kendisi çıkarmak zorunda kalır.
  - Push model: gözlenen, gözleyicilere değişiklik hakkında ayrıntılı bilgi gönderir. Birden fazla `update` metodu gerekebilir ve subject'in kodu karmaşıklaşır.
  - Aynı türden gözlenenlerin gözleyicileri ortaksa gözleyiciler `static` bir yapıda tutulabilir. Bir gözleyici birden fazla nesneyi gözleyebilir; farklı türden değişiklikleri farklı gözleyiciler izleyebilir (Java Swing'deki dinleyici mekanizması).
  - Tetikleme: gözlenen tetiklerse peş peşe değişiklikler çok sayıda mesaja yol açar. Göz-

leyiciler tetiklerse tetikleme komutu unutulabilir, değişiklik kaçırılabilir, sık sorgulama haberleşme masrafı yaratır.
  - Kullanım: bir soyutlamanın birbirine bağımlı birden fazla parçası varsa; bir nesnedeki değişiklik derleme anında bilinmeyen sayıda nesneyi etkileyecekse; bağlaşımı artırmadan diğer nesneler haberdar edilecekse.
- Observer örneği: yeni müşteri eklenmesi (ekrandan, 02:30:00, 02:32:00)
  - Yeni müşteri eklenince hoş geldin e-postası gönderilmeli, adres doğrulanmalı; sonradan, adres fiziksel bir şubeye 10 km'den yakınsa indirim kuponu eklenmesi istenir.
  - Sabit kodlu ilk çözüm yerine Observer: `Customer` (`static Vector<ICObserver> myObs`, `static attach/detach`, `getState()`, `notifyObs()`), `«interface» ICObserver` (`update(myCust : Customer)`), gerçeklemeler `AddrVerification`, `BrickAndMortar`, `WelcomeLetter`.
  - Yalnız müşteriler izleneceği için soyut `Subject` üst sınıfı yazılmadı. `notifyObs()` her gözleyiciye `update(this)` çağırır; hoca bunu bütün durum bilgisini gönderen bir push model olarak açıkladı.

Hocanın Özellikle Vurguladığı Kısımlar

- Kalıbın değeri, altyapıya yeni bir firma eklenince istemci kodunun değişmemesindedir
  - Iterator örneğinde iki çözüm aynı çıktıyı verir. Fark altyapıdadır: kalıpla garson tek bir `Menu` tipiyle çalışır, üçüncü bir menü için yalnız bir satır eklemek yeter. Acemi çözümde yeni alan, yeni döngü ve yinelenen kod gerekir.
  - Hoca, uzun `printMenu` metodunun bir public ve bir private metoda ayrılmasının ileride Extract Method refactoring'ine örnek olduğunu söyledi.
- Memento'nun asıl zorluğu sarmalamayı korumaktır
  - Durumu bir listede saklayıp geri dönmek için kalıba gerek yoktur. Kalıp, mementonun yalnız originator tarafından okunup yazılabildiği sıkı bir denetim istendiğinde anlamlıdır.
  - Bunun yolu dile göre değişir (C++'ta `friend`, Java'da paket düzeyi ya da `public` iç sınıf ile `private` metotlar).
  - Durum klonlanmazsa memento ile nesne aynı durumu paylaşır ve geri dönüş çalışmaz.
- Observer'da gerçekleme kararları ihtiyaca göre verilir
  - Pull ya da push, tetiklemeyi kimin yapacağı ve ne sıklıkla yapılacağı iyi analiz edilmezse gerekenden kötü bir çözüm üretilir. Hoca, haberleşme masrafının önemli olduğu IoT senaryolarını örnek gösterdi.
- Arayüz adlarını `I` ile başlatmak iyi bir alışkanlıktır; şemalarda arayüz kendini daha iyi belli eder.

Kısa Tekrar Notları

- Iterator: dolaşımı veri yapısından ayırır. İkinci çözümde istemci yalnız soyut liste ve soyut gezgin bilir.
- Java'nın `Iterator`'ı kalıbın basitleştirilmiş hâlidir; yetiyorsa KISS gereği o kullanılır.
- Gerçekleme soruları: dolaşırken değişikliğe izin var mı (yoksa klon), elemanlar tek tek mi dizi hâlinde mi, sıra nasıl belirlenir (gerekirse Strategy).
- Head First örneği: `Menu.createIterator()`, `DinerMenuIterator` (dizi), `PancakeHouseMenuIterator` (`ArrayList`), `Waitress.printMenu(Iterator)`.
- Memento: `Originator` (`createMemento`, `setMemento`), `Memento` (durum), `Caretaker` (saklar, dokunmaz).
- Java'da en sıkı çözüm: `Memento` `Originator`'ın `public` iç sınıfı, metotları `private`, durum klonlanır.
- Sahte memento sorunu: kimlik numarası denetimi ya da mementoları originator'da tutmak.
- Undo/redo örneği: `states` listesi ve `position`; konumun sınırları denetlenmeli.
- Observer: `attach`, `detach`, `notify` → her gözleyicide `update`.
- Pull: az bağlaşım, gözleyici işi çok. Push: ayrıntılı bilgi, subject karmaşık.
- Tetikleme gözlenende olursa çok mesaj, gözleyicide olursa unutma ve kaçırma riski var.
- Müşteri örneği: `static Vector<ICObserver>`; `Vector` iş parçacığı güvenlidir (thread-safe),

`ArrayList` ve `LinkedList` değildir.

Detaylı Açıklamalar

- Iterator kalıbında iki çözümün farkı
  - Birinci çözümde `ListIterator` doğrudan `List`'e bağlıdır. Veri yapısı ile dolaşım ayrı sınıflardadır, ama her yeni yapı için istemcinin farklı bir gezgin tipini bilmesi gerekir.
  - İkinci çözümde `AbstractList`'in `CreateIterator()` fabrika metodu, somut listeye uygun somut gezgini üretir. İstemci `AbstractList` ve `Iterator` arayüzleri üzerinden çalışır; `SkipList` gibi yeni bir yapı eklenince istemci değişmez.
- Head First menü örneğinde gezginlerin ayrıntısı (ekrandan, 00:20:00, 00:40:00)
  - `DinerMenu` en çok `MAX_ITEMS = 6` elemanlık bir `MenuItem[]` dizisi tutar; dolunca "Sorry, menu is full! Can't add item" yazar. `createIterator() new DinerMenuIterator(menuItems)` döndürür.
  - `DinerMenuIterator`: `next()` geçerli elemanı alıp konumu bir artırır. `hasNext()`, konum dizi boyunu aşmışsa ya da o konum `null` ise `false` döndürür (dizi tam dolmamış olabilir).
  - `remove()`: konum 0 ya da daha küçükse "en az bir `next()` yapmadan eleman silemezsin" iletisiyle `IllegalStateException` fırlatır. Aksi hâlde `position-1`'den sonraki elemanları bir geri kaydırır ve son hücreyi `null` yapar.
  - `PancakeHouseMenuIterator` `ArrayList` üzerinde çalışır: `hasNext()` için `size()` kullanır, `remove()` işi listenin `remove` metoduna iletir.
  - Arka plandaki menü sınıfları neredeyse hiç değiştirilmedi; amaç refactoring değil, iki altyapının ortak bir dolaşım arayüzüyle bütünleştirilmesidir.
- Memento ve Java'daki klon sorunu (ekrandan, 01:30:00)
  - Durum sınıfı `State` tek bir `String durum` alanı tutar. İç sınıf sürümünde `clone()` yeni bir `State` oluşturup `durum`'un kopyasını yazar ve onu döndürür:

    ```
    public State clone() {
        State s = new State();
        s.durum = new String(durum);
        return s;
    }
    ```

  - Hoca `clone()`'u `return this` yapınca caretaker'ın sonunda "yeni durum" yazdığını gösterdi: originator'ın değişikliği mementodaki aynı nesneyi de değiştirmişti. Hoca bunu Java'da nesnelerin referans değeriyle aktarılmasına bağladı.
  - `Cloneable` hiçbir metodun gerçeklenmesini istemeyen bir işaretleyici arayüzdür (marker interface); `clone()` `Object` sınıfından gelir. Hoca Java'nın önerisine uyarak `implements Cloneable` yazılmasını önerdi.
  - Ders sonrası not: `Cloneable`'ın denetimi yalnız `Object.clone()` (`super.clone()`) çağrıldığında yapılır. Yukarıdaki gibi `new` ile kopya üreten bir `clone()` için büyük olasılıkla zorunlu değildir, ama yazılması zarar vermez.
- Observer örneğinde neden `Vector` (derste sözlü, 02:33:41)
  - Java koleksiyonları iş parçacığı güvenli olan ve olmayan diye ikiye ayrılır. Ortak veriyi kullanan iş parçacıkları arasında yarış koşulu (race condition) oluşabilir.
  - Hocanın örneği: bir müşteri nesnesi gözleyicilere kendini gönderirken başka bir nesne müşterinin adresini değiştirirse, `AddrVerification` eski adresi, `BrickAndMortar` yeni adresi görebilir. Çok iş parçacıklı çalışmada bildirimin gözleyicilere hangi sırayla ulaşacağının garantisi yoktur.
  - Java'nın temel mekanizması `synchronized` anahtar kelimesiyle kilitlemedir. Bu güvenlik yavaşlık getirir; bu yüzden güvenli olmayan yapılar (`ArrayList`, `LinkedList`) daha hızlıdır. Çok iş parçacıklı uygulamada güvenli bir yapı seçilmelidir.
  - Concurrent yapıların kullanılıp kullanılamayacağı sorusuna hoca, kullanılan platformun sağladığı bütün yapıların rahatça kullanılabileceğini söyledi.
  - Ders sonrası not: Hoca `Vector`'ü "thread safe bir linked list" diye tanımladı; doğrusu, `Vector` dizi tabanlı, metotları `synchronized` olan büyüyebilen bir listedir (`ArrayList`'in senkronize karşılığı gibi).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.